

```
Microsoft OLE DB Provider for SQL Server (0x80040B14)

Column 'products.prodName' is invalid in the select
list because it is not contained in an aggregate
function and there is no GROUP BY clause.

/products.asp, line 13
```

Figure 3: SQL error caused by query string injection

```
SELECT * FROM Users WHERE usr=' ' or 1=1 -- AND password=' '
```

Because a pair of hyphens designates the beginning of the comment in SQL Server's T-SQL, the effective query is now:

```
SELECT * FROM Users WHERE usr=' ' or 1=1
```

For readers who don't know SQL, this translates to, "where the user field is blank, OR 1=1". The trick is that for the logical OR condition, this will always evaluate to *True*, since one of the operands to the OR statement is *True*. Thus, this query returns multiple user records, which validates the malicious login!

It doesn't stop there, however: since most Web applications have their 'administrator' user account added to the *Users* table as the first thing during development, the ASP code in the login page sees that record as the first returned record, and assumes it is the admin user logging in! A person, who doesn't even know a valid username and password, is given administrative permissions in your Web application...

## SQL injection via query string (URL)

An attacker might also attempt an SQL injection attack by adding SQL to the URL. How does that work? Let's look closely at an example. Assume a database with table and rows as created and populated by this SQL:

```
CREATE TABLE Products
(
  ID INT identity(1,1) NOT NULL,
  prodName VARCHAR(50) NOT NULL,
)

INSERT INTO PRODUCTS (prodName) VALUES ('Dell laptops')
INSERT INTO PRODUCTS (prodName) VALUES ('Nokia express music')
INSERT INTO PRODUCTS (prodName) VALUES ('Samsung dual sim range')
```

Let's also assume that we have the following ASP script, called *products.asp*, on the site; it uses the database with the above table.

```
<%
dim prodId
prodId = Request.QueryString("productId")

set conn = server.createObject("ADODB.Connection")
set rs = server.createObject("ADODB.Recordset")
query = "select prodName from products where id = " & prodId
conn.Open "Provider=SQLOLEDB; Data Source=(local); Initial Catalog=myDB;
User Id=sa; Password="
rs.activeConnection = conn
rs.open query
if not rs.eof then
response.write "Got product " & rs.fields("prodName").value
else
response.write "No product found"
end if
%>
```

If we visit *products.asp* in our browser with the (normal) URL: <http://www.example.com/products.asp?productId=1> then we will see the result is "Got product Dell laptop". The parameter is taken directly from the query string (submitted URL) and concatenated to the WHERE clause of the query, so the query generated by passing *productId=1* in the URL is:

```
SELECT prodName FROM products WHERE id = 1
```

Now, if the attacker tampers with the URL and submits something like <http://www.example.com/products.asp?productId=0 having 1=1> then the constructed query becomes:

```
SELECT prodName FROM products WHERE id = 0 or 1=1
```

This would produce an error as shown in Figure 3, or something similar. You can also see the error exposing the *products* fields such as *products.prodName*. The attacker can use this information maliciously, to insert or delete data from the table. For example, here is a sample malicious query injection:

```
http://localhost/products.asp?productId=0;INSERT INTO products(prodName)
VALUES(left(@@version,50))
```

Basically, it returns "No product found"; however, it may also run an INSERT query, adding the first 50 characters of SQL Server's @@version variable (which contains the details of SQL Server's version, build, etc) as a new record in the *Products* table. The attacker can then use this information to research specific exploits for that version of SQL Server.

Many programmers might suggest that they use double quotes instead of single quotes for security, but this is only